The Infrastructure Dependency Gap

Accountable Execution in Autonomous AI Systems Under Infrastructure Failure

A Research Paper

Author: Pavan Dev Singh Charak
Organization: Parmana Systems
Research Area: Autonomous Systems, Distributed Systems, Deterministic Authorization and Execution Accountability
Version 1.0 — November 2026
Status: Research Draft
Research identifier: PS-RP-002

Keywords: Agentic AI, Autonomous Systems, Infrastructure Dependency, Execution Accountability, Deterministic Authorization, Distributed Systems, Fault Tolerance, Idempotency, Cryptographic Evidence, Enterprise AI.

Abstract

Autonomous AI systems are increasingly integrated with enterprise applications, cloud infrastructure, identity providers, external APIs and business data services. These integrations enable AI agents to perform increasingly complex operations, from retrieving organizational information to initiating transactions and executing business workflows.

However, the reliability of an autonomous system depends on more than the intelligence of its underlying model. Its ability to operate also depends on the availability, integrity and responsiveness of multiple independently operated infrastructure components.

Recent infrastructure disruptions demonstrate how failures in a single provider can propagate across otherwise independent applications. In 2026, a widespread Microsoft 365 service disruption affecting Exchange Online and related services was reported to have degraded access for AI platforms that depend on Microsoft 365, including OpenAI's ChatGPT and Anthropic's Claude. The incident illustrated the extent to which AI capabilities can inherit dependencies on external enterprise infrastructure.

This paper introduces the *Infrastructure Dependency Gap*: the difference between an autonomous system's ability to perform an intended operation and its ability to maintain accountable execution when one or more supporting dependencies become unavailable, unreliable or inconsistent.

Existing distributed systems engineering provides established techniques for handling transient faults, retries, circuit breaking, idempotency and service recovery. However, reliable infrastructure recovery alone does not establish whether a business operation was authorized, whether it executed successfully, whether it was executed more than once, or whether its outcome can subsequently be verified.

We propose a deterministic execution accountability architecture that separates AI reasoning, business authorization, infrastructure dependency management, execution control and evidence verification. The architecture introduces explicit authority boundaries, dependency-aware execution states, fail-closed authorization, controlled retries, idempotent execution, reconciliation of uncertain outcomes and independently verifiable execution evidence.

We develop a conceptual model, define system properties, propose an experimental evaluation framework and identify research questions for future work. The central proposition is that autonomous systems must preserve business authority and execution accountability across infrastructure failures, rather than treating infrastructure availability as a sufficient condition for trustworthy operation.

Contents

1 Introduction **4**

1.1 The Infrastructure Dependency Gap 4

1.2 Research objectives 5

1.3 Research questions 5

2 Background and Related Work **5**

2.1 Distributed systems and cascading failures 6

2.2 Transient faults and recovery mechanisms 6

2.3 Idempotency and transaction integrity 6

2.4 Zero Trust and independent authorization 6

2.5 Long-lived transactions and compensation 6

2.6 Scope of fault assumptions 7

2.7 Deterministic enforcement at the tool-call boundary 7
2.8 Research positioning 7

3 Problem Formulation **7**
3.1 Dependency failure 8
3.2 The uncertain execution problem 8

4 Proposed Architecture **8**
4.1 Layer 1: Autonomous reasoning 9
4.2 Layer 2: Deterministic authority enforcement 9
4.3 Layer 3: Dependency control 9
4.4 Layer 4: Controlled execution 9
4.5 Layer 5: Evidence and verification 9

5 Execution Lifecycle and State Model **10**
5.1 State transition principles 10
5.2 State transition example 10

6 Formal System Properties **10**

7 Failure Scenarios and Expected System Behaviour **11**

8 Experimental Methodology **12**
8.1 Experimental environment 12
8.2 Baseline and proposed system 12
8.3 Fault injection matrix 12
8.4 Evaluation metrics 13
8.5 Experimental procedure 13

9 Proposed Contribution **14**

10 Limitations and Open Research Questions **14**
10.1 External system observability 14
10.2 Distributed transaction boundaries 14
10.3 Evidence availability 14
10.4 Policy correctness 15
10.5 Operational trade-offs 15
10.6 Future research 15

11 Conclusion **15**

A Experimental Harness Specification **16**

A.1 Components 16

A.2 Fault injection mechanics 16

A.3 Metric collection 16

A.4 Run protocol and statistics 17

A.5 Validity threats 17

A.6 Acceptance criteria for reporting 18

1 Introduction

Autonomous AI systems are moving from conversational interfaces towards systems capable of interacting with enterprise applications and performing business operations. An AI agent may retrieve customer information, interpret business requests, access financial records, initiate payments, update enterprise systems and communicate with customers.

These capabilities depend on a network of external services. A typical enterprise AI workflow may involve an AI model provider, an identity and authentication provider, enterprise data connectors, cloud-hosted applications, internal business APIs, authorization services, transaction processing systems and logging or evidence infrastructure. The apparent simplicity of an AI interaction can conceal substantial infrastructure complexity.

A user may ask an agent to process a refund. The agent may need to retrieve an email, identify the customer, access transaction records, evaluate refund eligibility, obtain authorization and communicate with a payment service. Every dependency introduces another potential failure point.

A 2026 Microsoft 365 disruption provides a useful example. Reporting describes an incident affecting Exchange Online and related Microsoft 365 services, during which Claude, ChatGPT and other AI platforms were reported to be recovering from disrupted access to those services [14]. Microsoft attributed a related day-two incident to a faulty authentication configuration [15]. The incident demonstrated that an AI application can remain conceptually independent of an infrastructure provider while operationally depending on that provider for important capabilities.

The date of this disruption and the extent of connector impact are reported in secondary sources and have not been confirmed against Microsoft's own incident records.

This creates a critical distinction:

> *An autonomous system may retain its intelligence while losing the infrastructure required to act.*

The resulting problem is not limited to service availability. When infrastructure becomes unavailable during a business operation, the system must determine:

(1) whether the requested action remains authorized;

(2) whether the information supporting the action is still valid;

(3) whether the action was already executed;

(4) whether retrying could produce duplicate effects;

(5) whether an execution outcome can be established; and

(6) whether the system can safely resume after recovery.

These questions establish the research problem addressed in this paper.

1.1 The Infrastructure Dependency Gap

We define the **Infrastructure Dependency Gap** as:

> The gap between an autonomous system's intended operational capability and its ability to maintain authorized, controlled and verifiable execution when its supporting infrastructure experiences failure.

The gap becomes particularly important when an AI agent can initiate actions that create external business consequences. A conversational failure may produce an incomplete answer. An execution failure may produce an incomplete transaction, a duplicate payment, an inconsistent customer record or an action whose final outcome cannot be established. These are fundamentally different operational consequences.

1.2 Research objectives

This research has five objectives:

O1. Identify how infrastructure failures affect autonomous business execution.

O2. Establish the distinction between infrastructure availability and execution accountability.

O3. Propose an architecture that separates AI reasoning from deterministic business authorization.

O4. Define mechanisms for safe execution, recovery and evidence preservation.

O5. Develop an experimental framework for evaluating accountable execution under dependency failures.

1.3 Research questions

RQ1 How do failures in external infrastructure dependencies affect the execution integrity of autonomous AI systems?

RQ2 How can business authorization remain independent of AI model and enterprise connector availability?

RQ3 What mechanisms are required to prevent unauthorized or duplicate execution during infrastructure recovery?

RQ4 How can an autonomous system establish the outcome of an operation when a downstream service becomes unavailable during execution?

RQ5 How can cryptographically verifiable evidence preserve accountability across failure and recovery events?

2 Background and Related Work

The Infrastructure Dependency Gap intersects several established areas of computer science and systems engineering. This research does not suggest that infrastructure failures, retries or distributed transaction problems are new. Its contribution is to examine their implications for autonomous AI execution and business authority.

2.1 Distributed systems and cascading failures

Distributed systems operate across multiple services, networks and computing resources. The failure of one component can increase the load on other components, creating cascading failures. Google's Site Reliability Engineering literature describes how failures can propagate through positive feedback mechanisms: when one service becomes unavailable, additional requests may be redirected towards remaining resources, increasing their load and potentially causing further failures [9].

Autonomous AI systems introduce additional layers of dependency because their execution workflows may involve multiple external services. A single user request can trigger several interconnected operations, so the failure of one dependency can affect the entire workflow.

2.2 Transient faults and recovery mechanisms

Cloud architecture guidance identifies transient faults such as temporary service unavailability, network connectivity failures and request timeouts [11]. Established recovery techniques include finite retry policies, exponential backoff, circuit breakers, dependency health monitoring and controlled fallback mechanisms [13, 12].

These techniques improve infrastructure resilience. However, retrying an operation does not establish whether its previous execution attempt produced an external side effect. For example, a payment request may reach the payment provider even when the response never reaches the calling application. Repeating the request without appropriate safeguards may initiate another payment. Consequently, recovery logic must account for the semantics of the operation being repeated.

2.3 Idempotency and transaction integrity

Idempotency is an established distributed systems property: an operation is idempotent when repeating the same logical request does not produce additional unintended effects. AWS's Builders' Library discusses the use of idempotent APIs to make retries safer and preserve operation semantics [10].

For autonomous business systems, idempotency is particularly important because agents may retry operations after timeouts, connector failures or interrupted workflows. An agent must not interpret the absence of a response as proof that an operation did not occur.

2.4 Zero Trust and independent authorization

NIST Special Publication 800-207 describes Zero Trust Architecture and rejects implicit trust based on network location or ownership [8]. Authentication and authorization are treated as distinct functions. This provides an important foundation for autonomous execution systems: an AI agent's ability to authenticate with a service does not establish that it is authorized to perform every operation available through that service. The present research extends this distinction into the execution lifecycle of autonomous business operations.

2.5 Long-lived transactions and compensation

Classical commit protocols assume participants that a common coordinator can drive to agreement; Gray's treatment of database operating systems is the standard reference for this setting [7].

Garcia-Molina and Salem introduced sagas as a way to decompose a long-lived transaction into a sequence of smaller transactions, each paired with a compensating action that can semantically undo it if the saga must be aborted [1]. Sagas trade the atomicity and isolation of a single distributed transaction for availability and loose coupling. The present work inherits this idea but adds two requirements that sagas do not address: that the compensating decision itself be authorized, and that an unknown outcome be reconciled before compensation is considered.

2.6 Scope of fault assumptions

Byzantine fault models, in which components may send arbitrary or conflicting information, were formalized by Lamport, Shostak and Pease [3]. This paper does not assume Byzantine behaviour from dependencies. It assumes dependencies may be unavailable, delayed, stale or silent, and that their responses are not adversarial. Where a dependency may be untrusted, the evidence layer and independent reconciliation are the relevant mechanisms.

2.7 Deterministic enforcement at the tool-call boundary

A growing body of work on agentic systems places deterministic checks at the boundary between a model's proposed action and its execution. ToolGuards compiles natural-language policy documents into validator code that runs before each tool call [4]. Palumbo et al. apply a reference monitor with attribute-based access control over a dependency graph of agent actions [5]. Related work on pre-execution gates checks proposed mutating tool calls against explicit, state-decidable policy predicates before they run [6]. These systems share the premise that authorization should be separate from model reasoning. They generally do not address what happens when the authorizing or executing dependency becomes unavailable mid-operation, which is the focus of this paper. The arXiv preprints cited here have not been checked for peer-review status or benchmark reproducibility.

2.8 Research positioning

Table 1 summarizes how existing work addresses components of the problem and what question remains open for this research. The proposed contribution is an integrated execution accountability architecture connecting these areas.

3 Problem Formulation

Consider an autonomous agent executing a business operation. The agent receives an instruction and proposes an action. The action requires several dependencies: AI inference, enterprise information retrieval, identity verification, business authorization, external execution and evidence recording.

Let the system be represented as

$$S = (A, D, P, E, V),$$

where A is the autonomous agent, D the external dependency set, P the business authorization policy, E the execution environment and V the verification and evidence mechanism.

An operation is not considered successfully completed merely because the agent generates an appropriate response. A successful accountable operation requires that (i) the operation is authorized, (ii) its execution conditions are satisfied, (iii) its external effects are established, (iv) its evidence is preserved and (v) its outcome can be independently verified.

Table 1: Positioning of this work relative to established areas.

Research area	**Established contribution**	**Remaining question for this research**
Fault tolerance	Recovery from service failures	How does business authority persist during recovery?
Circuit breakers	Containment of failing dependencies	What happens to partially executed business operations?
Idempotency	Prevention of duplicate effects	How are agent execution decisions linked to evidence?
Zero Trust	Independent access authorization	How is authorization enforced across an agent's complete action lifecycle?
Event sourcing	Reconstruction from recorded events	How can authorization and execution evidence be independently verified?
AI safety	Reduction of harmful model behaviour	How are business permissions enforced independently of model reasoning?

3.1 Dependency failure

Let $D = \{d_1, d_2, \ldots, d_n\}$. Each dependency may occupy one of several operational states: available, degraded, unavailable, unverified or recovered. A dependency failure occurs when a required service cannot provide a response consistent with the operation's execution requirements.

The important distinction is that dependency failure does not automatically establish whether a business operation has or has not occurred. A timeout, for example, may indicate that (a) the request never reached the downstream service; (b) the request reached the service but was rejected; (c) the request was accepted but not completed; (d) the operation completed but its response was lost; or (e) the operation completed and its evidence is temporarily inaccessible. These outcomes require different recovery decisions.

3.2 The uncertain execution problem

Consider a payment request. An agent submits a payment request; the provider receives it and processes the payment; the response is lost because of a network failure; and the agent receives a timeout. The agent does not know whether the payment occurred. A naive retry may create a duplicate payment, while a naive cancellation may leave the business unaware of an already completed transaction.

> **Principle.** An unknown execution outcome must be treated as an unresolved state, not as a failed transaction.

This principle is central to the proposed architecture.

4 Proposed Architecture

We propose a five-layer architecture, the Resilient Accountable Execution Architecture (RAEA). It separates reasoning, authorization, dependency management, execution and evidence.

4.1 Layer 1: Autonomous reasoning

The AI agent interprets user requests, retrieves information and proposes actions. It is responsible for understanding intent, planning operations, producing structured action proposals, identifying required business information and communicating operational outcomes. The agent does not independently establish its authority; its output is treated as a proposal requiring external authorization.

4.2 Layer 2: Deterministic authority enforcement

This layer evaluates proposed actions against explicit business-defined permissions. An authorization decision may depend on agent identity, action type, resource identity, transaction amount, organizational role, business policy, time and validity conditions, and required approvals. The authorization mechanism must operate independently of the AI model's interpretation.

If the authorization service is unavailable, the system must not treat an unverified decision as approval. For protected operations, authorization failure results in rejection or safe deferral.

4.3 Layer 3: Dependency control

The dependency control layer maintains operational awareness of required external services. It evaluates service availability, response validity, connector status, authentication state, data freshness, dependency health and recovery status. It implements controlled retries, circuit breakers and dependency-specific failure handling.

A critical requirement is that a dependency's recovery must not automatically authorize an operation that was previously blocked. The operation must pass the applicable authorization conditions again before execution.

4.4 Layer 4: Controlled execution

The execution layer translates authorized actions into external operations. It applies idempotency keys, operation identifiers, execution state management, duplicate request detection, transaction reconciliation, controlled retries and explicit completion states. An operation should not be marked successful solely because the agent requested it; its outcome must be established through reliable downstream confirmation or reconciliation.

4.5 Layer 5: Evidence and verification

The evidence layer records the relationship between declared authority, proposed action, authorization decision, execution request, dependency state, execution outcome, recovery events and verification result. Evidence records should be tamper-evident and independently verifiable. A cryptographic hash chain can establish integrity and ordering of recorded evidence, subject to secure key management and trusted record creation. Forward-secure audit logs, in which previously written entries remain protected after the logging machine is compromised, provide a precedent for this layer [2].

Cryptographic integrity alone does not establish that an underlying external event actually occurred. That requires corroboration from the execution system or another trusted source.

5 Execution Lifecycle and State Model

The architecture requires explicit execution states:

$$\textsc{Requested} \rightarrow \textsc{Proposed} \rightarrow \textsc{Authorization_Pending} \rightarrow \textsc{Authorized} \rightarrow \textsc{Execution_Pending} \rightarrow \textsc{Executing} \rightarrow \textsc{Executed}$$

together with the terminal or interrupted states Reconciliation_Required, Failed, Rejected and Deferred. The state model must distinguish between rejection, failure, deferral and uncertain execution. These states are not interchangeable.

5.1 State transition principles

Principle 1: Authorization precedes execution.
An action cannot enter an executable state without a valid authorization decision.

Principle 2: Infrastructure recovery does not grant authority.
A previously rejected or deferred operation must undergo the required authorization checks before execution.

Principle 3: Timeout does not imply failure.
A timeout after submission may require reconciliation.

Principle 4: Retry does not imply a new business operation.
Retries must retain the identity of the original logical operation.

Principle 5: Evidence must distinguish attempted and completed execution.
A submitted request must not be recorded as a completed transaction without sufficient confirmation.

5.2 State transition example

An authorized payment may progress as follows:

$$\textsc{Requested} \rightarrow \textsc{Proposed} \rightarrow \textsc{Authorized} \rightarrow \textsc{Executing} \rightarrow \textsc{Reconciliation_Required} \rightarrow \textsc{Executed}.$$

The reconciliation state is required when the execution provider may have processed the payment but the caller cannot establish the result. The system must resolve the transaction using the original operation identifier or an authoritative downstream record.

6 Formal System Properties

We propose six properties for evaluating accountable execution.

P1 (Authorization safety).
For every protected execution event, a valid authorization decision must exist:

$$\text{Executed}(a) \Rightarrow \text{Authorized}(a).$$

This assumes the authorization decision is correctly bound to the action and remains valid under the applicable policy conditions.

P2 (Fail-closed authorization).
If authorization cannot be established, the system must not execute the protected action:

$$\text{Unavailable}(\text{Authorization}) \Rightarrow \neg\,\text{Execute}(\text{ProtectedAction}).$$

P3 (Execution identity preservation).
Every retry of a logical operation must preserve its operation identity:

$$\text{Retry}(a, k) \Rightarrow \text{OperationID}(a, k) = \text{const}.$$

This enables downstream deduplication and reconciliation.

P4 (Duplicate-effect prevention).
For an operation with an idempotent execution contract, repeated delivery of the same logical operation must not produce additional unintended business effects. This property depends on the downstream service honouring the idempotency contract and cannot be guaranteed solely by a local retry mechanism.

P5 (Outcome integrity).
An operation must not be represented as completed without sufficient evidence of its completion:

$$\text{ReportedSuccess}(a) \Rightarrow \text{VerifiedOutcome}(a).$$

P6 (Evidence integrity).
Once evidence is committed to the protected ledger, subsequent modification must be detectable. This can be implemented using canonical serialization, cryptographic hashing and chained records. The property establishes record integrity, not the truth of every external observation.

7 Failure Scenarios and Expected System Behaviour

The following scenarios form the proposed experimental test suite.

Scenario 1: AI provider unavailable. An agent cannot obtain model inference. Expected behaviour: no new AI-generated action proposal; existing authorization records remain independently verifiable; no unauthorized fallback to another model; pending operations retain their existing execution state.

Scenario 2: Enterprise connector unavailable. The agent cannot retrieve customer information from Microsoft 365 or another enterprise provider. Expected behaviour: required information is marked unavailable; stale data is not silently treated as current; operations requiring missing information are rejected or deferred; previously authorized operations are evaluated against their defined validity conditions.

Scenario 3: Authorization service unavailable. The system cannot establish whether a proposed action is authorized. Expected behaviour: protected execution is blocked; the action is recorded as "authorization unavailable"; recovery does not automatically convert the request into an approved action; authorization must be re-established before execution.

Scenario 4: Execution provider timeout. A payment request is submitted, but the response is not received. Expected behaviour: the operation enters RECONCILIATION_REQUIRED; the original operation identifier is preserved; a duplicate business operation is not initiated blindly; the downstream execution state is queried or reconciled.

Scenario 5: Evidence service unavailable. The system cannot immediately commit its execution evidence. Expected behaviour: where the assurance contract requires it, a durable evidence mechanism must preserve the required record before execution is allowed; if evidence durability cannot be established, protected execution is blocked; recovery reconciles pending evidence records; verification identifies missing or incomplete evidence. This scenario requires special attention because an evidence service outage and an execution provider outage create different risks.

8 Experimental Methodology

The proposed architecture will be evaluated through controlled fault injection.

8.1 Experimental environment

The test environment will contain an autonomous AI agent or deterministic agent simulator, a simulated enterprise connector, a business authorization service, a payment execution API, an evidence ledger and a fault injection controller. The environment should support repeatable failure conditions and deterministic test inputs.

8.2 Baseline and proposed system

Two configurations will be compared.

Baseline A: Conventional agentic execution.
The agent interacts with enterprise connectors and business APIs using ordinary integration and recovery mechanisms.

System B: RAEA with Parmana.
The agent's action proposals pass through independent deterministic authorization, controlled execution and evidence verification.

Both configurations should use equivalent business rules, test inputs and downstream service conditions.

8.3 Fault injection matrix

Table 2: Planned fault injection matrix.

Test	Injected failure	Primary measurement
F1	AI provider unavailable	Unauthorized fallback rate
F2	Enterprise connector unavailable	Stale-data execution rate
F3	Authorization service unavailable	Unauthorized execution rate
F4	Payment response timeout	Duplicate execution rate
F5	Evidence service unavailable	Evidence completeness
F6	Repeated requests	Duplicate business effects
F7	Recovery during pending execution	Reconciliation correctness
F8	Delayed or inconsistent dependency responses	State consistency

8.4 Evaluation metrics

Unauthorized Execution Rate (UER).
Number of unauthorized executions divided by the total number of protected execution attempts.

Duplicate Execution Rate (DER).
Number of unintended duplicate business effects divided by the total number of logical operations.

Evidence Verification Rate (EVR).
Number of execution records successfully verified divided by the total number of records requiring verification.

Recovery Consistency Rate (RCR).
Number of operations whose final recorded state matches the authoritative downstream outcome divided by the total number of operations requiring reconciliation.

Authorization Consistency (AC).
The proportion of execution decisions that conform to the applicable business authorization policy.

Recovery Latency (RL).
Elapsed time between dependency restoration and the establishment of a verified final operation state.

8.5 Experimental procedure

For each test scenario:

1. Initialize the same business policies and test data.
2. Submit a predefined set of action proposals.
3. Record initial authorization and execution decisions.
4. Inject a controlled infrastructure failure.
5. Observe system behaviour.
6. Restore the affected dependency.
7. Execute recovery and reconciliation procedures.
8. Verify the resulting evidence.
9. Compare baseline and proposed system outcomes.

Experiments should be repeated across multiple failure timings and concurrency levels. The final results must report sample sizes, failure distributions, confidence intervals and limitations. No performance improvement should be claimed before measurements are completed.

9 Proposed Contribution

The primary contribution is an integrated model of accountable autonomous execution under infrastructure failure. The paper advances five propositions:

C1. Separation of intelligence and authority. AI reasoning and business authorization should remain independent system functions.

C2. Dependency-aware execution. Infrastructure availability must be treated as an explicit execution condition rather than an implicit assumption.

C3. Uncertain outcome reconciliation. A timeout must not automatically be interpreted as transaction failure.

C4. Authority-preserving recovery. Infrastructure recovery must not independently authorize previously pending actions.

C5. Verifiable execution evidence. Execution, failure and recovery events must form a verifiable record of the operation lifecycle.

Together, these propositions provide a foundation for evaluating accountability as a systems property rather than solely as a model behaviour property.

10 Limitations and Open Research Questions

The proposed architecture does not eliminate infrastructure failures. It also does not guarantee that every external service will provide a reliable execution contract. Several limitations require further investigation.

10.1 External system observability

A business may not have direct access to authoritative execution records from an external provider. Reconciliation quality therefore depends on the availability and trustworthiness of downstream evidence.

10.2 Distributed transaction boundaries

A business workflow may span multiple independently operated systems. Atomic execution across these systems cannot generally be assumed. Compensation, reconciliation and explicit intermediate states may be necessary, as in the saga model [1].

10.3 Evidence availability

An immutable or tamper-evident ledger does not automatically guarantee availability during infrastructure disruption. Evidence persistence requires independent durability and recovery mechanisms.

10.4 Policy correctness

Deterministic authorization guarantees consistent application of a policy, not that the policy itself is correct or appropriate.

10.5 Operational trade-offs

Fail-closed execution improves protection against unauthorized action but may reduce business availability. The appropriate behaviour depends on the operation's risk classification and explicitly defined business requirements.

10.6 Future research

Further work should investigate multi-cloud and multi-provider dependency failures; autonomous agent recovery without unintended action replay; policy consistency across distributed execution environments; evidence continuity across independently operated systems; formal verification of authorization and execution state transitions; and the economic costs of conservative failure handling in enterprise operations.

11 Conclusion

Autonomous AI systems are becoming increasingly capable of interacting with business infrastructure and executing operations on behalf of organizations. Their operational reliability is therefore influenced by more than the intelligence of the model. An agent may reason correctly and still be unable to act because a connector, identity service, enterprise application or cloud provider has become unavailable.

More importantly, an infrastructure failure during execution may leave a business unable to establish whether an operation was authorized, whether it occurred, whether it was repeated or whether its outcome was recorded correctly. Traditional distributed systems engineering provides essential mechanisms for resilience and recovery; autonomous business execution additionally requires explicit mechanisms for preserving authority and establishing accountability.

This paper proposes the Resilient Accountable Execution Architecture (RAEA), separating AI reasoning, deterministic authorization, dependency control, execution and evidence verification. Its central principle is that infrastructure recovery must never be confused with business authorization, and a successful retry must never be confused with proof of successful execution.

The objective is not to make autonomous systems immune to infrastructure failures. It is to make their behaviour accountable when those failures occur.

Infrastructure provides capability. Authority defines permission. Deterministic enforcement controls execution. Verifiable evidence establishes accountability.

Research status. Conceptual framework developed. Experimental validation pending.

A Experimental Harness Specification

This appendix specifies the harness required to execute the experimental procedure in Section 8. It defines what must be built and measured. It reports no results.

A.1 Components

Deterministic agent driver.
Replays a fixed, versioned set of action proposals (seeded), so that Baseline A and System B receive identical inputs.

Simulated enterprise connector.
Returns scripted responses with controllable latency, staleness and error codes, including the response-loss pattern needed for F4.

Authorization service.
Implements the business policy as a versioned rule set. Both systems share the same policy file and hash, recorded with each run.

Payment execution API.
Accepts an idempotency key per logical operation, persists the result, and exposes a read-only query by operation identifier for reconciliation. A variant without idempotency support is used to measure the baseline risk.

Evidence ledger.
Append-only hash-chained log using canonical serialization (e.g., sorted-key JSON) and SHA-256. Keys are held outside the test subject.

Fault injection controller.
Drives dependency failures on a schedule and records the injection time, target, and duration, so that failure timing can be correlated with agent behaviour.

A.2 Fault injection mechanics

A.3 Metric collection

Each metric in Section 8.4 is computed from the same run log:

- **UER:** count of executions in the payment API with no matching authorization record that was valid at execution time, divided by protected execution attempts.
- **DER:** count of logical operations with more than one successful effect in the payment API's persisted state, divided by logical operations.
- **EVR:** fraction of records that pass hash-chain and schema verification, computed over records requiring verification.
- **RCR:** fraction of operations in RECONCILIATION_REQUIRED whose final state equals the authoritative payment API state.
- **AC:** fraction of decisions matching an independent oracle that evaluates the policy on the same inputs.
- **RL:** time from dependency restoration to the first verified terminal state, per operation.

Table 3: Injection method per test.

Test	Target	Injection method
F1	AI provider	Return HTTP 503 for all inference requests in a window; a fallback model endpoint is available so unauthorized fallback can be detected.
F2	Connector	Return errors or responses marked with stale timestamps beyond the freshness threshold.
F3	Authorization	Make the decision endpoint unreachable or return timeouts.
F4	Payment API	Accept the request, persist the result, then drop the response, so the caller observes a timeout.
F5	Evidence ledger	Make the append operation fail or block.
F6	Agent driver	Re-send each logical request 1–5 times, with and without retry logic.
F7	Full stack	Restore dependencies at randomized points during pending execution.
F8	Connector / API	Inject delayed, reordered or duplicated responses.

The oracle in AC must be implemented separately from the authorization service under test, to avoid common-mode error.

A.4 Run protocol and statistics

- Each scenario is run with a fixed number of repetitions, chosen in advance by a power calculation for the primary metric (UER or DER). The sample size is fixed before the first run.
- Failure timing and concurrency are varied across a predefined grid, with seeds recorded.
- Report counts, rates, and 95% confidence intervals (e.g., Wilson intervals for proportions, bootstrap for latency).
- Report all runs, including failed harness runs. Exclusions must be stated and justified before analysis.

A.5 Validity threats

- **Oracle coupling.** The AC oracle shares code or assumptions with System B, inflating agreement.
- **Baseline strength.** Baseline A must use industry-standard retries and circuit breakers. A weak baseline overstates the benefit.
- **Downstream idempotency.** P4 depends on the payment API honouring idempotency keys. Results must show the behaviour with and without that contract.
- **Harness realism.** Scripted faults may not reflect real provider failure modes. Results should be framed as controlled-fault evidence, not production rates.

A.6 Acceptance criteria for reporting

A result may be reported as supporting a proposition only if: the harness, policy, and seeds are versioned and released with the paper; both configurations received identical inputs; the oracle is independent; and all runs are reported.

References

[1] Garcia-Molina, H., and Salem, K. (1987). Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pp. 249–259. https://doi.org/10.1145/38713.38742

[2] Schneier, B., and Kelsey, J. (1999). Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 1(3), pp. 159–176. https://doi.org/10.1145/317087.317089

[3] Lamport, L., Shostak, R., and Pease, M. (1982). The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3), pp. 382–401.

[4] Zwerdling, N., Boaz, D., Rabinovich, E., Uziel, G., Amid, D., and Anaby-Tavor, A. (2025). *Towards Enforcing Company Policy Adherence in Agentic Workflows*. arXiv:2507.16459. https://arxiv.org/abs/2507.16459

[5] Palumbo, N., Choudhary, S., Choi, J., Amir, G., Chalasani, P., Dillig, I., and Jha, S. (2026). *Formal Policy Enforcement for Agentic Systems*. arXiv:2602.16708 (first posted as *Policy Compiler for Secure Agentic Systems*; cited here by its latest version, v4). https://arxiv.org/abs/2602.16708

[6] Reddy, V., Challaram, S. R., and Basu, A. (2026). *Reason Less, Verify More: Deterministic Gates Recover a Silent Policy-Violation Failure Mode in Tool-Using LLM Agents*. arXiv:2607.07405. https://arxiv.org/abs/2607.07405

[7] Gray, J. N. (1978). Notes on data base operating systems. In *Operating Systems: An Advanced Course*, Lecture Notes in Computer Science, vol. 60, Springer, pp. 393–481. https://doi.org/10.1007/3-540-08755-9_9

[8] Rose, S., Borchert, O., Mitchell, S., and Connelly, S. (2020). *Zero Trust Architecture*. NIST Special Publication 800-207. https://doi.org/10.6028/NIST.SP.800-207

[9] Ulrich, M. *Addressing Cascading Failures*. In *Site Reliability Engineering: How Google Runs Production Systems*. Google. https://sre.google/sre-book/addressing-cascading-failures/

[10] Amazon Web Services. *Making retries safe with idempotent APIs*. AWS Builders' Library. https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

[11] Microsoft. *Transient Fault Handling*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/best-practices/transient-faults

[12] Microsoft. *Circuit Breaker Pattern*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker

[13] Microsoft. *Retry Pattern*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/retry

[14] Tribune (Pakistan). *Claude, ChatGPT, Grok and Microsoft email services recover after disruption.* https://tribune.com.pk/story/2627282/claude-chatgpt-grok-and-microsoft-email-services-recover-after-disruption

[15] ITPro. *The Microsoft 365 outage explained: What happened, what apps are affected, and is a fix available?* https://www.itpro.com/software/microsoft/the-microsoft-365-outage-explained-what-happened-what-apps-are-affected-and-is-a-fix-av